

Report: Gator Leaderboard System

Structure:

LeaderBoard:

- Contains functions and variables for leaderboard, the structure used by main to produce output from input file
- Functions:
 - InsertPlayer(score, player_id)
 - Checks player_id in hash table to make sure it is unique
 - Ensures player_id is exactly 6 characters
 - Inserts player and score into hash table and AVL tree
 - $O(\log n)$ complexity, $O(1)$ for hash table and $O(\log n)$ for AVL tree
 - RemovePlayer(player_id)
 - Checks to ensure player_id exists in hash table
 - Delete player from hash table and AVL tree
 - $O(\log n)$ complexity, $O(1)$ for hash table and $O(\log n)$ for AVL tree
 - AddScore(score, player_id)
 - Checks that player exists in hash table
 - Deletes player from AVL tree, then inserts player back with updated score
 - Increase hash table value at player_id by score
 - $O(\log n)$ complexity, $O(1)$ for hash table changes and $O(\log n + \log n)$ for AVL tree
 - GetRank(player_id)
 - Checks that player exists in hash table
 - Uses rank function of tree to get rank
 - $O(\log n)$ complexity, $O(\log n)$ for getting rank of player
 - GetPlayerWithRank(rank)
 - Checks that rank is valid
 - Uses select function of tree to get Node of player with rank passed in
 - $O(\log n)$ complexity, $O(\log n)$ for getting player with rank passed in
 - PrintTopK(k)
 - Ensures k is valid
 - If $k > n$ then use n
 - Uses select function to get the top k players
 - $O(k \log n)$ complexity, $O(\log n)$ for each select function times k for the amount of times it is called
 - Leaderboard()
 - Initializes leaderboard by creating empty tree and hash table

- $O(1)$ average complexity, unless already previously called then it must delete previous tree in $O(n)$ time

AVLTree:

- Contains functions and variables used for creating and maintaining AVL tree
- Function:
 - AVLTree()
 - Creation of AVL tree, sets root=nullptr
 - $O(1)$ complexity
 - ~AVLTree()
 - Destructor of AVL tree, uses delete tree called at root to delete whole tree
 - $O(n)$ complexity, has to go through all nodes, uses post order traversal to delete all nodes
 - Insert(score, player_id)
 - Inserts node of given player_id and score into AVL tree
 - Create retrace stack to be used by Update
 - $O(\log n)$ complexity, $O(\log n)$ to insert then $O(\log n)$ to retrace and update tree
 - Delete(score, player_id)
 - Deletes node of given player_id and score
 - Creates retrace stack to be used by UpdateDelete
 - $O(\log n)$ complexity, $O(\log n)$ to find and delete node then $O(\log n)$ to retrace and update tree
 - SearchByKey(score, player_id)
 - Searches tree to find node with score and player_id
 - $O(\log n)$ complexity
 - Rank(score, player_id)
 - Retrieves rank of node with passed in player_id and score
 - Uses node sizes to keep running total of rank
 - $O(\log n)$ complexity, to search and find node of given rank all rank updating operations are constant
 - Select(rank)
 - Retrieves node with given rank
 - Uses node sizes to check and update rank of node while traversing
 - $O(\log n)$ complexity, to search and find node of given rank
 - GetSize()
 - Helped function, used to retrieve size of tree, returns size of root
 - $O(1)$ complexity
 - DeleteTree(node)
 - Recursive deletion of node and its subtree, uses post order traversal to delete left and right children before deleting node
 - Used by destructor
 - $O(n)$ complexity, n is the size of the original node to start deleting from
 - GetRoot()
 - Return root

- $O(1)$ complexity
- Update(stack, node)
 - Retraces path traveled by insertion, updates size and checks balance of each node
 - Calls rotate functions when their criteria is met
 - $O(\log n)$ complexity, same complexity as insertion since it is moving along the same path just in opposite direction
- UpdateDelete(stack)
 - Retraces path traveled by deletion, updates size and checks balance of each node
 - Calls rotate functions when their criteria is met
 - $O(\log n)$ complexity, same complexity as delete since it is moving along the same path just in opposite direction
- LeftLeft(node)
 - LeftLeft rotation case, rotates around passed in node
 - Updates pointers and size, returns node to be made new child of the next node in retrace path
 - $O(1)$ complexity, just pointer reassignment
- RightRight(node)
 - RightRight rotation case, rotates around passed in node
 - Updates pointers and size, returns node to be made new child of the next node in retrace path
 - $O(1)$ complexity, just pointer reassignment
- LeftRight(node, node)
 - LeftRight rotation case, uses RightRight rotation around first node passed in then LeftLeft rotation around second
 - Updates pointers and size, returns node to be made new child of the next node in retrace path
 - $O(1)$ complexity, just pointer reassignment
- RightLeft(node, node)
 - RightLeft rotation case, uses LeftLeft rotation around first node passed in then RightRight rotation around second
 - Updates pointers and size, returns node to be made new child of the next node in retrace path
 - $O(1)$ complexity, just pointer reassignment
- CalculateBalance(node)
 - Calculates balance of passed in node using height
 - Used for updating tree after insertion/deletion
 - Returns calculated balance factor
 - $O(1)$ complexity
- CheckHeight(node)
 - Helper function
 - Checks height of node, if node is null return 0
 - Returns height of node
- CheckSize(node)
 - Helper function
 - Checks size of node, returns this value, if node is null return 0
- GetLargestLeft(node, stack)

- Returns largest element in the left subtree of passed in node, used for deletion of a node with two children
- Updates retrace stack as well to be used in UpdateDelete
- $O(\log n)$ complexity

Node:

- Contains Node structure for AVL tree
- Nodes contain player_id, score, size, left and right child pointers, and height
- Function:
 - Node(score, player_id)
 - Constructor for Node
 - Sets size to 1, score and player id to passed in values, left and right child to null, and height to 1
 - $O(1)$ complexity

Main program:

The main file reads the designated input file, then determines the correct function of the leaderboard to invoke based on the input, all output is piped to the output file which can be viewed after main execution. Error handling is implemented in cases of no input file, input file cannot be found, and incorrect commands being passed through input, along with all other error handling as mentioned above.